

GitHub Actions

GitHub Actions est un outil puissant intégré à GitHub qui vous permet d'automatiser de nombreuses tâches liées à votre développement logiciel, y compris l'exécution de vos tests unitaires à chaque modification de votre code.

Pourquoi utiliser GitHub Actions pour vos tests ?

- **Intégration continue (CI)** : À chaque fois que vous poussez du code sur votre dépôt GitHub, GitHub Actions peut déclencher automatiquement vos tests. Cela vous permet de détecter les erreurs et les régressions plus rapidement, avant qu'elles ne se propagent dans votre projet.
- **Gain de temps** : Fini les lancements manuels de tests ! GitHub Actions s'occupe de tout, vous laissant vous concentrer sur le développement.
- **Qualité du code** : En exécutant vos tests systématiquement, vous vous assurez que votre code reste fonctionnel et maintenable, même après des modifications importantes.
- **Collaboration facilitée** : Les résultats des tests sont visibles directement sur GitHub, ce qui facilite la communication et la collaboration entre les membres de l'équipe.

Comment automatiser vos tests avec GitHub Actions ?

1. **Créer un fichier de workflow** : Dans votre dépôt GitHub, créez un fichier `.github/workflows/tests.yml` (ou un nom similaire). Ce fichier YAML définit les étapes de votre workflow d'automatisation.
2. **Définir les déclencheurs** : Indiquez quand vous souhaitez que vos tests soient exécutés (par exemple, à chaque `push` sur la branche `main`, lors de la création d'une pull request, etc.).
3. **Configurer l'environnement** : Spécifiez l'environnement d'exécution de vos tests (système d'exploitation, version de Python, dépendances, etc.).
4. **Exécuter les tests** : Utilisez les actions appropriées pour installer vos dépendances, puis lancez vos tests avec la commande de votre framework de test (pytest, unittest, etc.).

5. **Afficher les résultats** : Utilisez les actions pour afficher les résultats des tests directement sur GitHub, par exemple en les publiant dans les commentaires de la pull request.

Exemple simple de workflow pour pytest :

```
name: Python application CI/CD

on:
  push:
    branches: [ main, dev ] # Branch to survey
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ["3.11"]

    steps:
      - uses: actions/checkout@v3
      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v3
        with:
          python-version: ${ matrix.python-version }
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements_train.txt
          # Add any specific installation commands for libcpab
          pip install -e libcpab
```

```
# tests

- name: Run unit tests (un fichier)

  run: |

    pytest tests/test_0.py

- name: Run unit tests (un autre fichier)

  run: |

    pytest tests/test_1.py
```

Débogage avec un fichier “__init__”

```
import os
import sys

sys.path.insert(0,
                os.path.abspath(
                    os.path.join(
                        os.path.dirname(__file__),
                        ".."
                    )
                ))

from modules.tools import add
```